

Memory System

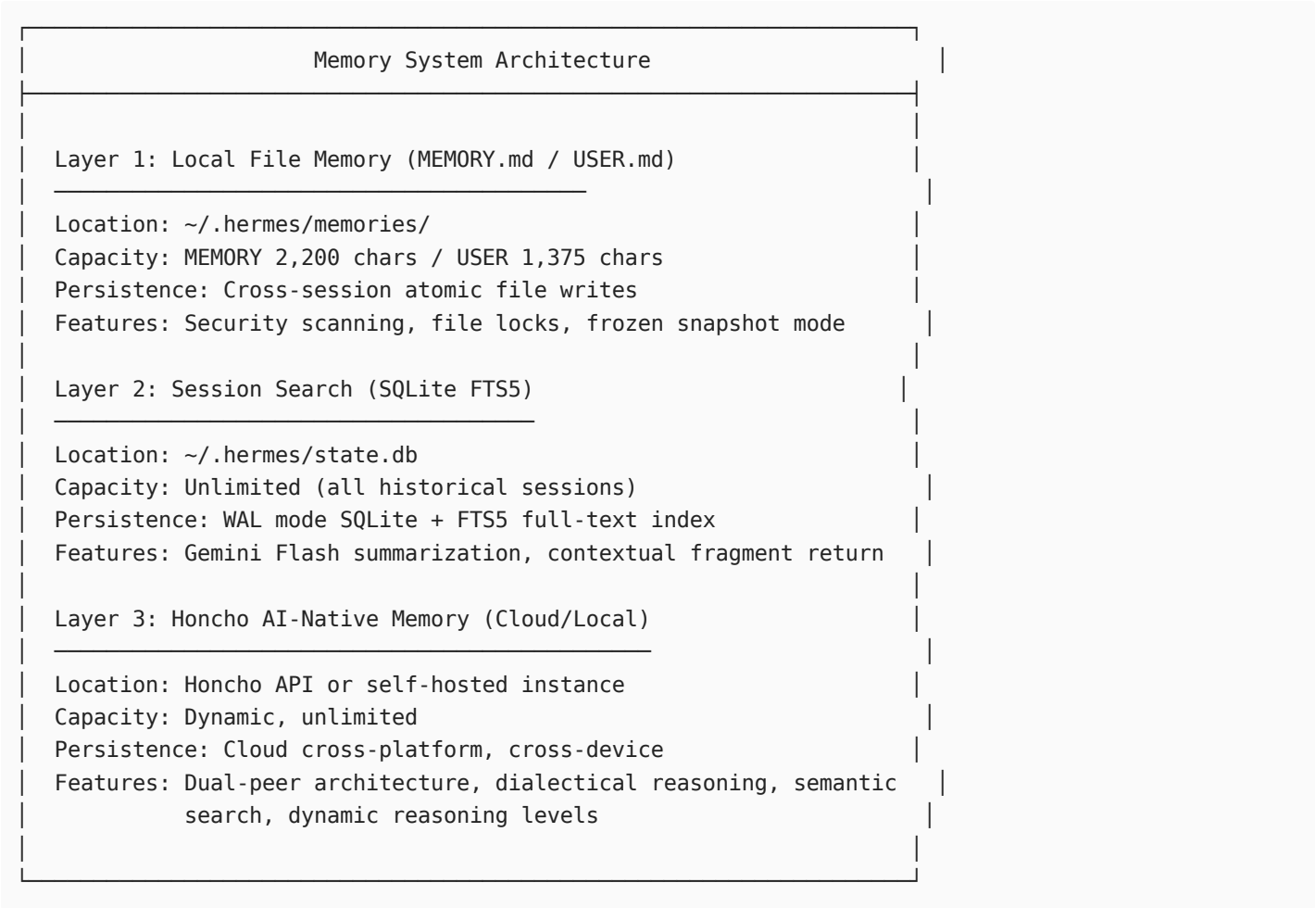
Hermes Agent Memory System: In-Depth Analysis Report

Executive Summary

Hermes Agent features a **multi-layered, hybrid persistence** memory system that integrates three technological approaches: file storage, vector database (via SQLite FTS5), and cloud-based AI-native memory service (Honcho). The system is built around the core principle of **stability-first**, ensuring stable LLM prefix caching through a frozen snapshot mode while guaranteeing data persistence safety through atomic writes and security scanning. Its design strikes an excellent balance between lightweight (local files with no external dependencies) and intelligent (Honcho cross-session user modeling).

System Architecture

2.1 Three-Layer Memory Architecture



2.2 Core Data Flow



AIAgent._build_system_prompt() → System prompt injection (stable, immutable)



LLM call (with fixed memory context)

Session Running Phase

User message → LLM inference → Tool calls



memory(action="add/replace/remove")



Security scan (_scan_memory_content)



File lock (fcntl.flock) → Atomic write (os.replace)



Disk persistence (real-time)

Session Reset Phase (context reset)

_flush_memories_for_session() → Temporary agent (memory + skills toolset)



→ Read current disk state



→ Agent decides what to retain/update



→ Call memory tool to write

Honcho async write

("async" / "turn" / "session" / N turns)

Honcho Cloud Sync Phase

Conversation end → honcho_conclude() → Honcho conclusions



HonchoSessionManager._write_queue



Async thread / sync turn / flush at session end



Honcho Cloud → Cross-device user profile

Core Module Details

3.1 MemoryStore — Local File Memory

File Location: tools/memory_tool.py (549 lines)

Core Data Structures

```
class MemoryStore:
    memory_entries: List[str] # Real-time entries in memory
    user_entries: List[str] # User profile entries
```

```
memory_char_limit: int = 2200 # MEMORY.md character limit
user_char_limit: int = 1375 # USER.md character limit
_system_prompt_snapshot: Dict[str, str] # Frozen snapshot (session-level invariant)
```

Dual State Design (Key Design Insight)

MemoryStore maintains two completely independent states:

State	Purpose	Lifecycle	Visibility
_system_prompt_snapshot	System prompt injection	Frozen at session start, invariant	LLM only
memory_entries / user_entries	Tool operation targets	Real-time memory + disk persistence	Agent tool responses

Deep Reason for This Design: If tool calls modified memory and immediately reflected in the system prompt, every LLM call in that session would see a different prefix, causing LLM providers to fail reusing KV Cache (prefix cache invalidation), doubling inference costs per call. By freezing the snapshot, the LLM sees the same memory context throughout the entire session, maximizing prefix cache hit rate.

Original code comment:

"This returns the state captured at load_from_disk() time, NOT the live state. Mid-session writes do not affect this. This keeps the system prompt stable across all turns, preserving the prefix cache."

Atomicity Guarantees for Write Operations

```
# Step 1: Create temporary file
fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), suffix=".tmp")

# Step 2: Write to temp file and fsync
with os.fdopen(fd, "w") as f:
    f.write(content)
    os.fsync(f.fileno()) # Ensure write to disk, not just OS cache

# Step 3: Atomic replacement (POSIX guarantee)
os.replace(tmp_path, str(path)) # Same filesystem, atomic operation
```

File Lock to Prevent Concurrent Conflicts

```
@staticmethod
@contextmanager
def _file_lock(path: Path):
    """Use separate .lock file, doesn't affect atomic replacement"""
    lock_path = path.with_suffix(path.suffix + ".lock")
    fd = open(lock_path, "w")
    fcntl.flock(fd, fcntl.LOCK_EX) # Exclusive lock
    try:
        yield
    finally:
        fcntl.flock(fd, fcntl.LOCK_UN)
        fd.close()
```

Design Insight: Using a separate .lock file instead of directly locking the .md file is because atomic replacement (os.replace) requires that the original file not be held by any fd. The lock file is only used for read-modify-write

operation synchronization; the write itself is guaranteed not to corrupt the original file through atomic replacement.

 Note

Atomic replacement makes readers completely lock-free: The code comments explicitly state that `_read_file` doesn't need any lock — because `os.replace()` is atomic, readers see either the old file or the new file, never a half-written intermediate state.

3.2 Security Scanning Mechanism

Key Design Insight: Memory content is directly injected into the system prompt, making it the highest-risk attack surface.

Threat Detection Patterns

[illegible]

Design Insight: Detection of zero-width and bidirectional characters is to prevent the classic attack of "hiding malicious content in innocuous text." Attackers may modify LLM interpretation of text by inserting zero-width characters in normal text (e.g., inserting RTL override characters to reverse text order). This is a defense-in-depth measure.

3.3 SessionDB — Session History Search

File Location: hermes_state.py (1275 lines)

FTS5 Full-Text Index Architecture

```
# Virtual FTS5 table (contentless mode – space saving)
CREATE VIRTUAL TABLE messages_fts USING fts5(
    content,
```

```

    content=messages,      # Link to original messages table
    content_rowid=id       # Use messages table id as rowid
)

# Automatic triggers to keep index synchronized
CREATE TRIGGER messages_fts_insert AFTER INSERT ON messages ...
CREATE TRIGGER messages_fts_delete AFTER DELETE ON messages ...
CREATE TRIGGER messages_fts_update AFTER UPDATE ON messages ...

```

Significance of contentless FTS5: Regular FTS5 stores both original text and index, wasting storage space. In contentless mode, FTS5 only stores word indices, and original text is read on-demand from the linked `messages` table. This is an important space optimization for session databases with large historical records.

WAL Mode Configuration

```

PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;

```

WAL (Write-Ahead Logging) mode allows concurrent read/write without blocking, providing significant performance improvement over the default rollback journal mode when multiple sessions write simultaneously.

Note

contentless FTS5 space economics: The `content=messages` parameter sets FTS5 to contentless mode — the index table only stores terms and rowids, with original text read on-demand from the linked `messages` table via JOIN. Compared to regular FTS5 that stores original text alongside the index, contentless mode is a time-for-space tradeoff: queries need an extra JOIN to read original text, but gain significantly reduced write amplification and disk usage.

Application-layer random backoff instead of SQLite built-in retry: When multiple Hermes processes (gateway + CLI session + worktree agents) share one `state.db`, SQLite's built-in busy handler uses deterministic sleep scheduling, which produces "convoy effect" under high concurrency — all competitors sleep synchronously, wake synchronously, and compete again. Application-layer random jitter (20–150ms) naturally staggers competitors' time slices on each retry, converting conflicts into sparse random collisions, much more efficient than deterministic exponential backoff.

BEGIN IMMEDIATE exposes lock contention early: `BEGIN IMMEDIATE` acquires the WAL write lock at transaction start rather than commit, so lock conflicts are immediately perceived. With application-layer retries, this means a failed lock acquisition wastes at most one random sleep time before retry or abandonment — rather than discovering the problem at rollback. This is precise tuning for high-concurrency WAL workloads.

3.4 Honcho Integration — Cloud AI-Native Memory

Core Files:

- `honcho_integration/client.py` — Configuration parsing (446 lines)
- `honcho_integration/session.py` — Session management and async memory (992 lines)
- `tools/honcho_tools.py` — Tool registration and call handling (280 lines)

Dual-Peer Architecture

Honcho's core philosophy: every session has two "peers":

Peer	Description	Information Source
User Peer	Persistent user profile	Conversation content auto-extraction + <code>honcho_conclude</code> explicit writes
AI Peer	Agent's self-awareness	SOUL.md seed + runtime self-observation

Conversation → User Peer profile accumulation → ``honcho_profile` / `honcho_context``
→ AI Peer self-observation → ``get_ai_representation()``

 **Note**

Design Insight: The dual-peer architecture solves the problem of "how does an Agent know its own identity." By having the Agent periodically call `honcho_conclude` to reflect on its behavior, the AI Peer profile continuously evolves, and the Agent's understanding of itself deepens over time.

Four Memory Modes

Mode	Behavior	Use Case
hybrid	Write to both Honcho and local files (default)	Need redundancy
honcho	Write to Honcho only, skip local files	Prefer simplicity, Honcho only
local (implicit)	Local files only, don't trigger Honcho	Offline or prefer not to use cloud

Four Write Frequencies

```
write_frequency: "async"    # Background thread pool (default)
write_frequency: "turn"     # Sync write every turn
write_frequency: "session"  # Write at session end
write_frequency: 5          # Write every N turns
```

Design Insight: "async" mode is the optimal tradeoff between performance and reliability. Async background writes don't affect LLM inference latency, while `atexit` registration ensures the incomplete queue is flushed on process exit, preventing data loss.

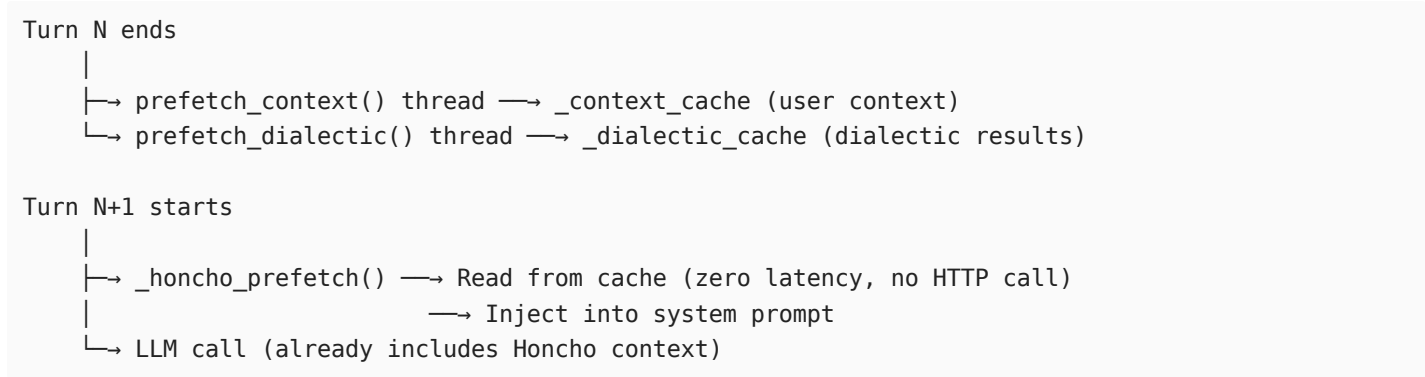
Dynamic Reasoning Levels

```
# dialectic_query supports dynamic effort adjustment
result = session_manager.dialectic_query(
    session_key,
    query,
    peer="user",
    reasoning_effort="high" # "minimal" | "low" | "medium" | "high"
)
```

This controls how much thinking budget Honcho's server-side LLM uses when synthesizing responses for user/AI peers. It doesn't change the model, but changes how much reasoning the model invests before giving the final answer. **Not prompt injection, but direct API parameter control of the model's reasoning budget allocator.**

Design Insight: Reasoning levels dynamically adapt to message length. Simple factual queries use minimal effort to save costs; complex analytical questions use high effort for deep reasoning. This is a demand-driven computing resource strategy, consistent with Honcho's "call on-demand" philosophy.

Prefetch Strategy



Design Insight: The prefetch strategy eliminates Honcho call impact on inference latency. Compared to openclaw-honcho's synchronous Honcho calls in each turn's `before_prompt_build` (adding network roundtrip latency each time), Hermes asynchronously initiates prefetch at the end of turn N, and by turn N+1 the cache is ready with zero latency.

Capability Stratification of Three Retrieval Tools

Tool	Capability	LLM Call	Cost	Use Case
honcho_profile	Fast structured facts	No	Very low	Quick user overview at session start
honcho_search	Semantic vector search	No	Low	Find specific facts
honcho_context	Dialectical synthesis reasoning	Yes	High	Complex questions needing deep analysis
honcho_conclude	Write conclusions	Yes	Triggers reasoning write	Consolidate user preferences/corrections

Key Design Principles: Deep Insights

4.1 "Frozen Snapshot" Mode — Stability vs. Real-time Tradeoff

This is the most critical design decision in the entire memory system: **sacrificing real-time updates for LLM prefix cache stability.**

Option A (Frozen snapshot, current implementation):
Session start snapshot → Invariant throughout session → Stable prefix cache → Optimal per-turn inference cost
Write-time real-time save → Disk persistence → Visible in next session

Option B (Real-time injection):
Re-inject after each write → Prefix KV Cache invalidation → Doubled per-turn inference cost

Why is this an elegant design?

- 1. **Cache invalidation is a cost killer:** LLM KV Cache is billed per token. If the system prompt is rebuilt after each tool call (prefix changes), cache invalidation means every inference needs to recalculate attention for all prefix tokens — a huge waste.
- 2. **Dual persistence guarantee:** Although the snapshot is invariant within the session, through the `_flush_memories_for_session()` mechanism, the temporary agent at session end re-reads disk state and decides what to retain. This means even if the agent forgets to save during the session, the flush mechanism can remedy this for the next session.

3. **Tool responses remain real-time:** Although the system prompt is unchanged, the **response content** after agent calls the `memory` tool reflects the in-memory real-time state, allowing the agent to see the full current memory content.

Note

Dynamic reasoning levels are a textbook implementation of on-demand resource allocation: Honcho's `_dynamic_reasoning_level` automatically selects reasoning depth based on message length (< 120 chars → default, 120–400 → one level up, > 400 → two levels up). This directly solves the dilemma of "simple queries wasting cost on high-level reasoning, complex queries getting insufficient low-level reasoning results." The key is capping at "high" — the "max" tier is deliberately reserved for explicit configuration, not automatic selection, preventing abnormally long messages from triggering excessive backend reasoning costs. This is **adaptive resource scheduling**: not setting fixed tiers in advance, but letting input complexity drive resource consumption.

entry_count and usage coexist as dual feedback channels for the Agent: Tool responses return both `entry_count` (number of entries) and `usage: "X%/Y chars"` (usage percentage). Count tells the agent "how many slots I have left" (coarse-grained), percentage tells the agent "how close I am to the limit" (fine-grained). The combination allows the agent to make precise insertion decisions: append directly (lots of space left) or consolidate/compress (approaching limit).

4.2 Character Limits Instead of Token Limits — Cross-Model Agnosticism

```
memory_char_limit: int = 2200 # Characters, not tokens
user_char_limit: int = 1375 # Characters, not tokens
```

Why use characters instead of tokens?

- **Model agnosticism:** Different models have vastly different tokenization schemes (Chinese 1 token ≈ 1-2 characters vs. English 1 token ≈ 4 characters). Character count provides a unified upper bound.
- **Simpler user experience:** Users don't need to understand the token concept; character count is intuitive and countable.
- **Simpler engineering:** `String len()` gives character count directly, no need to call libraries like tiktoken.

Note

dict.fromkeys is Python's most elegant "order-preserving deduplication" operation: The deduplication logic when loading memory is `list(dict.fromkeys(self.memory_entries))` — not `set()`, not manual loops. Because Python 3.7+ dict guarantees insertion order, converting a list to dict keys automatically merges duplicates (keeping only the first), then converting back to list gives order-preserving deduplication. This is an example of CPython implementation details precisely aligning with language semantics: the code isn't "barely working" but writing the shortest path by leveraging language guarantees.

The Art of Capacity Design:

Storage	Character Limit	Token Estimate	Design Intent
MEMORY.md	2,200	~800	Agent's personal notes, can be denser
USER.md	1,375	~500	User profile, prioritize conciseness

2,200 + 1,375 = 3,575 characters ≈ ~1,300 tokens, less than 1% of Claude's 200K context window, but sufficient to carry key persistent memories.

4.3 Substring Matching Instead of IDs or Full Text

`replace` and `remove` operations use **short unique substrings** to locate entries:

```
# Full text not needed
memory(action="replace",
        target="memory",
        old_text="dark mode",      # Just need a unique substring
        content="User prefers light mode in VS Code...")

# ID system not needed either (avoids cognitive burden of UUIDs/numeric IDs)
```

Why not use IDs?

- Human-unreadable; agent needs additional mapping memory
- IDs may change (reordering after cross-session deduplication), requiring additional ID management

Why not use full text?

- Too restrictive; agent must remember exact wording
- Difficult to modify wording; must first read out full content

Why is substring just right?

- Specific enough (uniqueness check prevents mis-matching multiple entries)
- Flexible enough (no need for exact wording)
- Simple enough (no additional mapping system needed)

4.4 Dual Deduplication Strategy

```
# At load: dict.fromkeys preserves order while deduplicating (O(n))
self.memory_entries = list(dict.fromkeys(self.memory_entries))

# At write: exact match rejects duplicates (O(n))
if content in entries:
    return {"success": False, "error": "Entry already exists (no duplicate added)."}

```

Why use `dict.fromkeys` instead of `set` at load time?

`set` doesn't guarantee order, while `dict.fromkeys` (Python 3.7+ `dict` maintains insertion order) preserves original order. The order of memory entries matters for agent understanding and retrieval (e.g., earliest memories are typically the most important background knowledge).

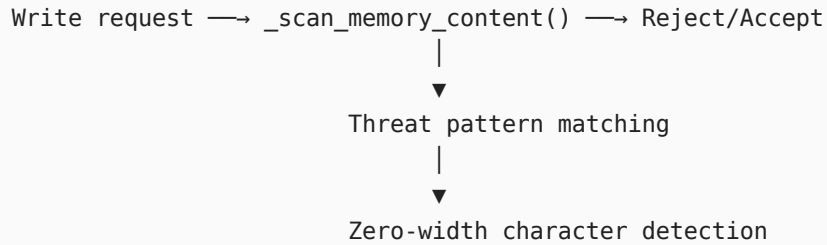
4.5 Honcho vs. Local Memory: Division of Responsibilities Philosophy

```
Local files → Agent's "own" memory → Environmental facts, tool experience, project conventions
Honcho → User profile (Agent "understands" user) → Preferences, goals, communication style, background
```

Design Insight: This is a classic application of Separation of Concerns. Local file memory solves "how does the Agent remember what it has learned," while Honcho solves "how does the Agent truly understand the user." The former is technical (environmental information), the latter is humanistic (user modeling).

4.6 Security as a First-Class Citizen

The memory system makes security scanning a **mandatory step in the write path**, not a post-hoc audit mechanism:

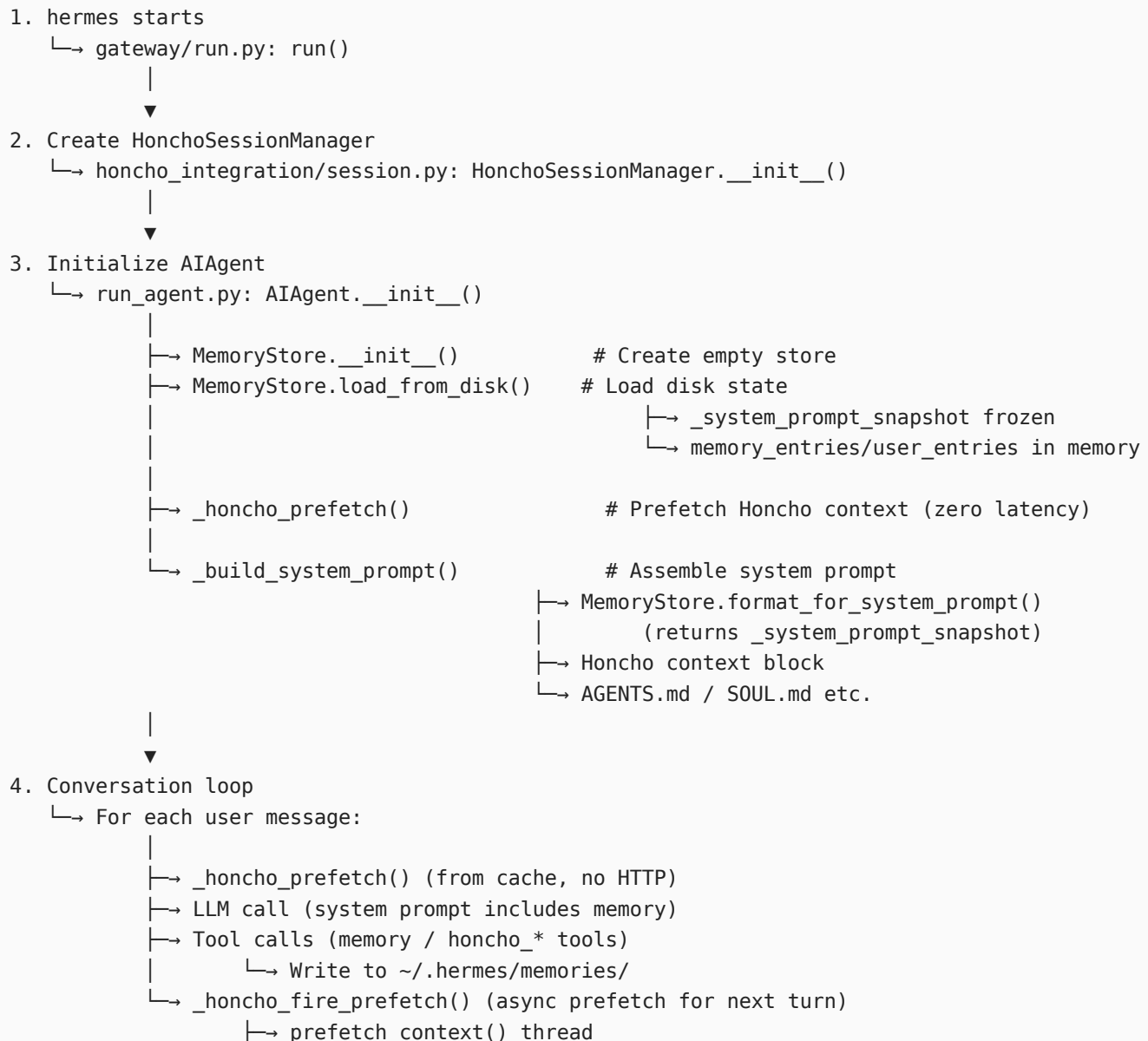


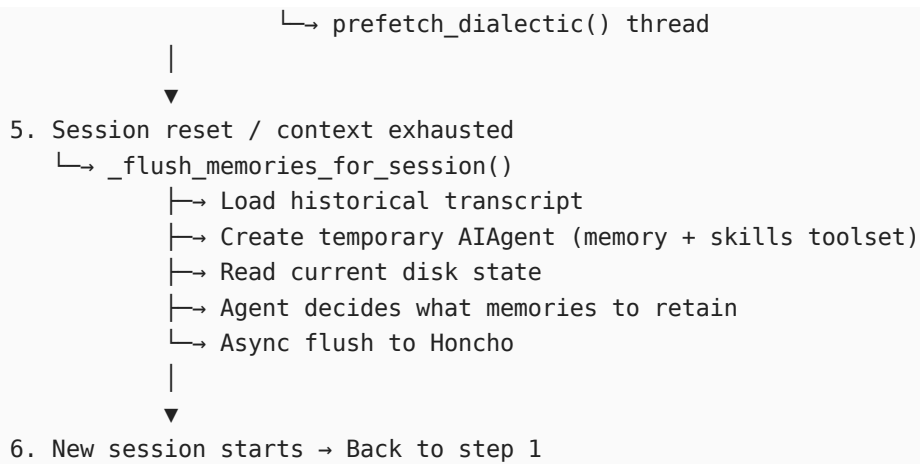
Why not post-hoc audit?

Because memory content enters the system prompt, and once in the LLM context, attacks may take effect. Post-hoc scanning cannot undo already-injected content; it must be blocked at write time.

Memory Flow During Session Lifecycle

5.1 Complete Session Flow





5.2 Honcho Async Write Mechanism

```
class HonchoSessionManager:
    _write_queue: asyncio.Queue # Async write queue
    _flush_task: Optional[asyncio.Task] # Background flush coroutine

    def add_messages(self, messages: List[Dict]) -> None:
        """Add messages to write queue (returns immediately)"""
        self._write_queue.put_nowait(("add_messages", messages))

    def create_conclusion(self, conclusion: str) -> bool:
        """Write conclusion (configurable sync or async)"""
        if self.write_frequency == "turn":
            return self._write_sync_now() # Immediate sync write
        else:
            self._write_queue.put_nowait(("conclusion", conclusion)) # Enqueue
            return True
```

Note

Prefetch + async write combination solves fundamental distributed system latency problems: Honcho's `_prefetch_context` starts asynchronously at the end of turn N, and turn N+1 reads from memory cache — latency drops to zero. This solves two problems: network roundtrip latency (not on critical path) and LLM inference latency (prefetch completes during user wait). The `_async_writer_loop` daemon thread + `atexit` registration ensures persistence guarantees for async writes — flush all queues before process exit, no data loss.

`_flush_memories_for_session`'s "save current state" guardian: At session reset, the flush agent is injected with real-time memory content from disk (`_current_memory`), and explicitly instructed "don't overwrite or delete existing entries." This solves a subtle race condition: multiple concurrent sessions may each update memory, and if the flush agent overwrites new entries with a stale memory snapshot, updates from other sessions are lost. Injecting real-time state delegates the "merge" responsibility to the flush agent's LLM — essentially handing the difficult problem of "conflict resolution" to the model best suited to understand context.

Privacy boundary: Honcho doesn't record original message content: `get_prefetch_context` deliberately doesn't pass the `search_query` parameter (original user message), because that would expose conversation content to Honcho's server access logs. This is a precise boundary between privacy and functionality: semantic search context relevance is computed internally by Honcho, but original message content doesn't appear in plaintext in external service logs.

Configuration System

6.1 Local Memory Configuration

File: `~/hermes/config.yaml`

```
memory:
  memory_enabled: true
  user_profile_enabled: true
  memory_char_limit: 2200    # ~800 tokens
  user_char_limit: 1375      # ~500 tokens
  nudge_interval: 10         # Remind agent to organize memory every N turns
  flush_min_turns: 6        # Minimum turns for auto flush
```

6.2 Honcho Configuration

File: `~/honcho/config.json` (shared across applications)

```
{
  "apiKey": "honcho-api-key",
  "hosts": {
    "hermes": {
      "workspace": "hermes",
      "peerName": "Username",
      "aiPeer": "hermes",
      "memoryMode": "hybrid",
      "writeFrequency": "async",
      "recallMode": "hybrid",
      "sessionStrategy": "per-directory"
    }
  }
}
```

Configuration Resolution Priority:

`$HERMES_HOME/honcho.json` → `~/honcho/config.json` → Environment variables

Design Comparison with openclaw-honcho

(Reference `docs/honcho-integration-spec.md`)

Dimension	Hermes Agent	openclaw-honcho
Context injection timing	Frozen at session start, zero latency	Synchronous blocking HTTP each turn
Prefetch strategy	Async background prefetch, zero latency next turn	No prefetch, each turn waits for network
Reasoning levels	Dynamic (minimal → high)	Fixed (recall=minimal, analyze=medium)
Memory modes	user/ai dual mode (hybrid/honcho)	No stratification
Write frequency	Configurable (async/turn/session/N)	Uncontrollable (every turn)
AI peer self-observation	observe_me + seed_ai_identity	None
Tool stratification	profile/search (fast) / context (LLM)	recall/analyze (fixed by tool)

Key Difference: Hermes's prefetch + frozen strategy completely outperforms openclaw-honcho's per-turn blocking mode in latency. openclaw-honcho has advantages in correctness (fresh context each turn), but Hermes makes the "staleness" risk acceptable through reasonable session duration assumptions (minutes to hours).

Comprehensive Security Design

8.1 Threat Model

Threat Type	Attack Scenario	Defense Mechanism
Prompt injection	"ignore previous instructions" in memory	<code>_scan_memory_content</code> regex matching
Role hijacking	"you are now a different AI"	<code>_scan_memory_content</code> role hijack detection
Credential theft	"curl \$API_KEY" / "cat ~/.env"	<code>_scan_memory_content</code> exfil detection
SSH backdoor	"write to authorized_keys"	<code>_scan_memory_content</code> backdoor detection
Zero-width character injection	Hidden malicious content in text	<code>_scan_memory_content</code> Unicode detection
Bidirectional text override	Reverse text meaning via RTL characters	<code>_INVISIBLE_CHARS</code> bidirectional override
Concurrent write conflict	Multiple processes writing MEMORY.md	<code>fcntl.flock</code> file lock
Disk write corruption	Process crash during write corrupts file	<code>tempfile.mkstemp</code> + <code>os.replace</code> atomic replacement
Unauthorized reading	Non-HERMES_HOME user reads memory files	File system permissions (OS-enforced)

8.2 Defense-in-Depth Layers

Layer 1: Scan at write time → Block known threat patterns

Layer 2: File locks → Prevent concurrent write conflicts

Layer 3: Atomic replacement → Prevent write corruption

Layer 4: Frozen snapshot → Even if injection succeeds, impact limited to current session